Day 8: Search, Indexing & Information Retrieval

Understanding how large-scale systems organize and retrieve data efficiently.

Search Basics

- Indexing vs querying
- Fast retrieval systems

Inverted Index & Tokenization

- Term → Document mapping
- Tokenization
- Normalization

Indexing Pipeline & Updates

- Ingestion pipeline
- Near real-time indexing
- Document updates

Ranking & Relevance

- Term frequency
- Freshness
- Personalization

Indexing Pipeline & Updates

- Ingestion pipeline
- Near real-time indexing
- Document updates

Autocomplete & Distributed Search

- Prefix search
- Query suggestions
- Sharded & replicated indexes

Exact Match vs Full-Text Search

- Exact match finds precise terms
- Full-text finds any relevant text
- Results ranked by relevance

Sharding, Replication & Fan-Out

- Shard indexes
- Replication for availability
- Fan-out & merge results

Efficient search turns raw data into relevant information

Day 8: Search, Indexing, and Information Retrieval

Day 8 focuses on how large systems organize content so users can find the right documents, products, or records quickly.

It covers:

- search system basics
- inverted indexes and tokenization
- indexing pipeline and document updates
- ranking and relevance
- exact match vs full-text search
- autocomplete and prefix search
- sharding, replication, and query fan-out in search systems

In short, Day 8 is about turning large collections of data into something users can query efficiently and meaningfully.

Core ideas

Search basics

- A `search system` usually separates indexing from querying.
- During indexing, documents are parsed and transformed into a searchable structure.
- During querying, the system finds candidate documents and ranks them by relevance.

Inverted index and tokenization

- An `inverted index` maps each term to the documents that contain it.
- `Tokenization` decides how text is split into searchable words or terms.
- Search quality depends heavily on normalization choices like lowercasing, stemming, stop-word removal, and punctuation handling.

Indexing pipeline and updates

- Real systems often use an ingestion pipeline that validates, enriches, and indexes documents asynchronously.
- Updates are hard because search indexes must stay reasonably fresh without hurting query performance.
- Many systems use near-real-time indexing instead of fully synchronous updates.

Ranking and relevance

- Search is not only about finding matches; it is also about ordering them well.

- Relevance can include term frequency, field weights, freshness, popularity, personalization, and business rules.
- Poor ranking makes a technically correct search engine feel broken to users.

Autocomplete and distributed search

- `Autocomplete` often uses tries, prefix indexes, or cached popular prefixes.
- Large search systems shard indexes across machines and merge results from many shard responses.
- Replication improves availability, but it also creates consistency and freshness trade-offs.

Day-8 sample: Inverted Index Search

This repository includes a small inverted index search sample in both Python and Java.

Functional requirements

- Add documents to an index
- Tokenize document text into searchable terms
- Search for documents containing all query terms
- Keep the implementation simple enough for interview explanation

High-level components

- `Document store`: holds the original content
- `Tokenizer`: normalizes and splits text into terms
- `Inverted index`: maps terms to document IDs
- `Query processor`: intersects matching document sets and returns results

Data flow

1. A document is added to the system
2. The tokenizer normalizes and splits the text
3. Each term is written into the inverted index with its document ID
4. A user submits a query such as `distributed search`
5. The query processor finds matching posting lists and returns the documents that contain all query terms

Project structure

```
Day-8/
  README.md
  inverted-index-sample/
    python/
      inverted_index.py
      demo.py
    java/
```

```
InvertedIndex.java
Main.java
```

Run the sample

Python

```
cd inverted-index-sample\python
python demo.py
```

Java

```
cd inverted-index-sample\java
javac Main.java InvertedIndex.java
java Main
```

Interview takeaways

- Search systems are usually two systems in one: indexing and querying
- Inverted indexes are efficient, but tokenization and ranking decide quality
- Relevance is a product problem as much as an infrastructure problem
- Search freshness, shard fan-out, and query latency are common production trade-offs
- Autocomplete, typo tolerance, and ranking often matter more than raw matching speed

Next improvements

- Add TF-IDF or BM25-style scoring
- Add phrase search and fielded search
- Add stop-word filtering and stemming
- Add prefix search for autocomplete
- Extend the sample into a tiny document search API

Source Code Appendix

Day-8/inverted-index-sample/python/inverted_index.py

```python
from __future__ import annotations

import re
from collections import defaultdict
from dataclasses import dataclass, field


def tokenize(text: str) -> list[str]:
    return re.findall(r"[a-z0-9]+", text.lower())


@dataclass
class InvertedIndex:
    _documents: dict[str, str] = field(default_factory=dict)
    _postings: dict[str, set[str]] = field(default_factory=lambda: defaultdict(set))

    def add_document(self, document_id: str, text: str) -> None:
        if not document_id:
            raise ValueError("document_id must be non-empty")
        if document_id in self._documents:
            raise ValueError(f"document {document_id} already exists")

        self._documents[document_id] = text
        for term in set(tokenize(text)):
            self._postings[term].add(document_id)

    def search(self, query: str) -> list[tuple[str, str]]:
        terms = tokenize(query)
        if not terms:
            return []

        matching_ids: set[str] | None = None
        for term in terms:
            postings = self._postings.get(term, set())
            if matching_ids is None:
                matching_ids = set(postings)
            else:
                matching_ids &= postings

            if not matching_ids:
                return []

        assert matching_ids is not None
        return [(document_id, self._documents[document_id]) for document_id in sorted(matching_ids)]
```

Day-8/inverted-index-sample/python/demo.py

```python
from inverted_index import InvertedIndex


def main() -> None:
    index = InvertedIndex()

    index.add_document("doc-1", "Distributed search systems use inverted indexes for fast retrieval.")
    index.add_document("doc-2", "Distributed tracing helps debug request paths across services.")
    index.add_document("doc-3", "Search ranking decides which matching documents users see first.")
    index.add_document("doc-4", "An inverted index maps terms to the documents that contain them.")

    queries = [
        "distributed search",
        "inverted index",
        "ranking documents",
        "trace retrieval",
    ]

    for query in queries:
        print(f"query: {query}")
        results = index.search(query)
        if not results:
            print("  no matches")
            continue
```

```
        for document_id, text in results:
            print(f"  {document_id}: {text}")
        print()


if __name__ == "__main__":
    main()
```

Day-8/inverted-index-sample/java/InvertedIndex.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class InvertedIndex {
    private static final Pattern TOKEN_PATTERN = Pattern.compile("[a-z0-9]+");

    private final Map<String, String> documents = new HashMap<>();
    private final Map<String, Set<String>> postings = new HashMap<>();

    public void addDocument(String documentId, String text) {
        if (documentId == null || documentId.isBlank()) {
            throw new IllegalArgumentException("documentId must be non-empty");
        }
        if (documents.containsKey(documentId)) {
            throw new IllegalArgumentException("document already exists: " + documentId);
        }

        documents.put(documentId, text);
        Set<String> uniqueTerms = new HashSet<>(tokenize(text));
        for (String term : uniqueTerms) {
            postings.computeIfAbsent(term, ignored -> new HashSet<>()).add(documentId);
        }
    }

    public List<SearchResult> search(String query) {
        List<String> terms = tokenize(query);
        if (terms.isEmpty()) {
            return List.of();
        }

        Set<String> matchingIds = null;
        for (String term : terms) {
            Set<String> termPostings = postings.getOrDefault(term, Set.of());
            if (matchingIds == null) {
                matchingIds = new HashSet<>(termPostings);
            } else {
                matchingIds.retainAll(termPostings);
            }

            if (matchingIds.isEmpty()) {
                return List.of();
            }
        }

        List<String> sortedIds = new ArrayList<>(matchingIds);
        Collections.sort(sortedIds);

        List<SearchResult> results = new ArrayList<>();
        for (String documentId : sortedIds) {
            results.add(new SearchResult(documentId, documents.get(documentId)));
        }
        return results;
    }

    private static List<String> tokenize(String text) {
        Matcher matcher = TOKEN_PATTERN.matcher(text.toLowerCase());
        List<String> tokens = new ArrayList<>();
        while (matcher.find()) {
            tokens.add(matcher.group());
        }
```

```
        return tokens;
    }

    public record SearchResult(String documentId, String text) {
    }
}
```

Day-8/inverted-index-sample/java/Main.java

```
import java.util.List;

public class Main {
    public static void main(String[] args) {
        InvertedIndex index = new InvertedIndex();

        index.addDocument("doc-1", "Distributed search systems use inverted indexes for fast retrieval.");
        index.addDocument("doc-2", "Distributed tracing helps debug request paths across services.");
        index.addDocument("doc-3", "Search ranking decides which matching documents users see first.");
        index.addDocument("doc-4", "An inverted index maps terms to the documents that contain them.");

        runQuery(index, "distributed search");
        runQuery(index, "inverted index");
        runQuery(index, "ranking documents");
        runQuery(index, "trace retrieval");
    }

    private static void runQuery(InvertedIndex index, String query) {
        System.out.println("query: " + query);
        List<InvertedIndex.SearchResult> results = index.search(query);
        if (results.isEmpty()) {
            System.out.println("  no matches");
            return;
        }

        for (InvertedIndex.SearchResult result : results) {
            System.out.printf("  %s: %s%n", result.documentId(), result.text());
        }
        System.out.println();
    }
}
```